

Dual-Agent Debate Matchmaking System Based on Similarity and Diversity Strategies

Jyoti Triklani AU2544015
Gautam Prajapati AU2544033

Department - Masters in Computer Science Engineering
School of Engineering And Science (SEAS)
Ahmedabad University

21st November 2025

Course: CSE601 Computational Thinking
Instructor: Dr. Shristi Sharma

Outline

- 1 Problem Statement
- 2 Solution Overview
- 3 Literature Review
- 4 Computational Thinking Framework
- 5 System Design
- 6 Implementation
- 7 Results & Evaluation
- 8 Future Work
- 9 Conclusion
- 10 References

Problem Statement

Challenges in Current Debate Systems

- Online debate platforms often produce mismatched opponents
- Leads to unengaging or unbalanced discussions
- Single-agent algorithms fail to capture full range of perspectives
- Users experience frustration with poor debate pairings

Need for AI-Driven Solution

- Dynamic matching strategies required
- Combination of similarity and diversity approaches
- Multi-agent system for balanced arguments
- Automated evaluation for fair judgment

Our Solution: Dual-Agent System

System Components:

- **Agent Alpha:** Logical arguments
- **Agent Beta:** Emotional arguments
- **Moderator:** Automated judgment
- **Gemini API:** LLM backend
- **LangGraph:** Orchestration

Key Features:

- Automatic position assignment
- Multi-round debates
- Fair scoring mechanism

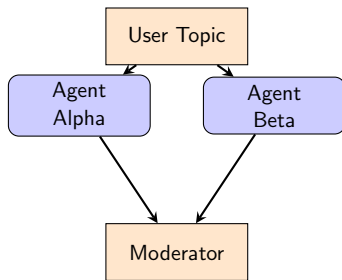


Figure: System Architecture

Literature Review

Foundational Research

- **Multi-Agent Systems [1]**
 - Wooldridge & Jennings (1995): Theory and practice of intelligent agents
 - Foundation for autonomous agent collaboration and competition
- **Computational Thinking [2]**
 - Wing (2006): Foundational framework for problem-solving
 - Four pillars: Decomposition, Pattern Recognition, Abstraction, Algorithms

Modern AI Technologies

- **Transformer Architecture & LLMs [3, 4]**
 - Vaswani et al. (2017): Attention mechanism for NLP
 - Brown et al. (2020): GPT-3 few-shot learning capabilities
- **Implementation Tools [6, 7, 8]**
 - LangGraph: Multi-agent workflow orchestration
 - Google Gemini API: Advanced language model
 - Streamlit: Interactive web interfaces

CT Pillar 1 & 2: Decomposition and Pattern Recognition

1. Decomposition

System broken into modules:

- Input Processing Module
- Agent Management (Pro, Con, Judge)
- Gemini API Integration
- Argument Generation Engine
- Scoring & Evaluation System
- Results Display Module

2. Pattern Recognition

Identified patterns:

- **Argument Structure:**
Claim → Evidence → Reasoning
- **Debate Flow:**
Opening → Rebuttals → Closing
- **Evaluation Criteria:**
Logic, Evidence, Relevance, Persuasiveness
- API response parsing patterns

CT Pillar 3 & 4: Abstraction and Algorithm Design

3. Abstraction

Hiding complexity:

- Agent behavior unified across all topics
- API complexity hidden behind simple methods
- Standardized argument format
- Consistent scoring system
- State management abstraction

4. Algorithm Design

Step-by-step process:

- 1 Initialize system components
- 2 Process user topic input
- 3 Generate Pro arguments
- 4 Generate Con arguments
- 5 Iterate for N rounds
- 6 Evaluate all arguments
- 7 Calculate scores
- 8 Declare winner

System Architecture

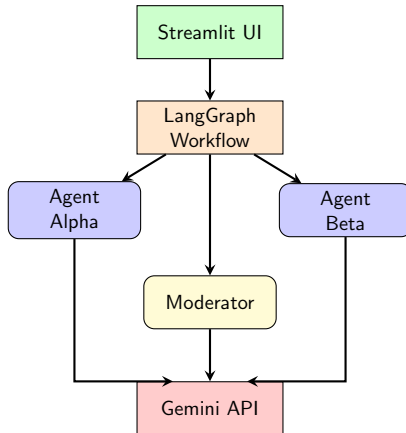


Figure: Multi-Layer System Architecture

Technology Stack & Architecture

Core Technologies:

- **Python 3.13**
 - Primary programming language
 - Rich AI/ML ecosystem
- **Streamlit 1.51**
 - Web-based user interface
 - Real-time updates
- **LangGraph 1.0.3**
 - Multi-agent orchestration
 - State management
- **Google Gemini API**
 - LLM for argument generation
 - Natural language understanding

Project Modules:

- **Agents Module:**
 - Agent personality prompts
 - Response generation logic
 - Position assignment
- **Graph Module:**
 - State definitions
 - Workflow implementation
 - Round management
- **Main Application:**
 - User interface
 - Input handling
 - Results display

Dual-Agent Design Strategy

Agent Alpha

Logical & Analytical Characteristics:

- Evidence-based arguments
- Statistical references
- Structured logic

Strategy:

- Factual data
- Logical frameworks

Agent Beta

Passionate & Persuasive Characteristics:

- Emotional appeal
- Real-world examples
- Persuasive rhetoric

Strategy:

- Compelling stories
- Human impact

Response Constraints

Agents: 200 words — **Moderator:** 250-300 words

LangGraph Workflow Implementation

Debate Flow Process:

① Initialize State

- Topic from user
- Empty message list
- Round counter = 0

② Agent 1 Response

- Reads topic & history
- Generates argument

③ Agent 2 Response

- Reads opponent's argument
- Generates counter-argument

④ Round Check

- If rounds remain → Repeat
- If complete → Moderator

State Management:

• Topic Tracking

- Maintains subject
- Consistent context

• Message History

- Stores all arguments
- Enables rebuttals

• Round Counter

- Tracks progress
- Controls termination

Final Step:

- Moderator evaluates
- Declares winner
- Provides justification

Automatic Position Assignment

Smart Topic Parsing

When users input topics with "vs", the system automatically assigns positions:

How it Works:

- Detects "vs" in topic
- Agent Alpha → Before "vs"
- Agent Beta → After "vs"
- Clear position indicators

Example Topics:

- "Hardwork vs Talent"
- "AI vs Human Intelligence"
- "City vs Village Life"

Example: Hardwork vs Talent

Agent Alpha:

Argues for **Hardwork**

Agent Beta:

Argues for **Talent**

Each agent maintains their position throughout all rounds

Key Features: Scoring & Rate Limiting

Scoring System

Evaluation Criteria:

- Argument strength (30%)
- Evidence quality (25%)
- Logical coherence (20%)
- Rebuttal effectiveness (15%)
- Presentation clarity (10%)

Outcome Categories:

- Clear Winner: > 3 points difference
- Tie: ≤ 3 points difference

Rate Limiting

Challenge:

- Free tier API limits
- 429 errors without delays
- Need for stable operation

Solution:

- 2-second delays between calls
- Prevents rate limit errors
- Ensures stable operation
- Smooth user experience

Streamlit User Interface

UI Components:

- **Topic Input:** Text field for debate subject
- **Round Selection:** Slider (1-5 rounds)
- **API Key Input:** Secure sidebar input
- **Position Display:** Shows each agent's arguing position
- **Real-time Streaming:** Live response generation

Color-Coded Display:

- Agent Alpha - Blue boxes (Logical arguments)
- Agent Beta - Pink boxes (Passionate arguments)
- Moderator - Orange boxes (Final judgment)

Agent Alpha

Agent Beta

Moderator

Sample Debate Example

Topic: "AI vs Human Intelligence"

Agent Alpha (AI Intelligence) - Score: 42/50

"AI systems process vast datasets instantly, achieving superhuman performance in pattern recognition, chess mastery, and medical diagnosis..."

Agent Beta (Human Intelligence) - Score: 45/50

"Human intelligence encompasses creativity, emotional depth, and moral reasoning that no algorithm can replicate..."

Moderator Verdict

Winner: Agent Beta (Human Intelligence)

Tie with nuanced arguments. Beta's emphasis on irreplaceable human qualities slightly edges Alpha's technical capabilities.

Implementation Challenges & Solutions

Technical Challenges

1. Authentication Issues

- LangChain auth errors
- **Solution:** Direct Google SDK

2. Model Availability

- Models not found
- **Solution:** Used Gemini 2.0 Flash

3. Rate Limiting

- 429 API errors
- **Solution:** 2-second delays

Design Challenges

1. Response Length

- Inconsistent token limits
- **Solution:** 200-word limit

2. Position Assignment

- Manual assignment tedious
- **Solution:** Automated "vs" parsing

3. UI Readability

- Poor color contrast
- **Solution:** Dark text, light backgrounds

Planned Features:

- **Download Transcript**
 - PDF export
 - TXT format
- **Multi-Agent Extension**
 - 3+ agents
 - Panel discussions
- **Voice Integration**
 - Text-to-speech
 - Audio debates

Research Directions:

- **Advanced Scoring**
 - ML-based evaluation
 - Fact-checking
- **User Participation**
 - Human-AI debates
 - Audience voting
- **Domain Specialization**
 - Legal debates
 - Scientific discussions

Conclusion

Key Achievements

- Successfully implemented dual-agent debate system
- Applied computational thinking principles
- Integrated Google Gemini API with LangGraph
- Created intuitive Streamlit interface
- Automated position assignment & judgment

Computational Thinking Application

Demonstrated all four CT pillars:

- **Decomposition:** Breaking debate into components
- **Pattern Recognition:** Argument structures
- **Abstraction:** State & agent design
- **Algorithm Design:** Workflow implementation

References I

-  Wooldridge, M., & Jennings, N. R. (1995). *Intelligent agents: Theory and practice*. The Knowledge Engineering Review, 10(2), 115-152.
-  Wing, J. M. (2006). *Computational thinking*. Communications of the ACM, 49(3), 33-35.
-  Vaswani, A., et al. (2017). *Attention is all you need*. Advances in Neural Information Processing Systems, 30.
-  Brown, T., et al. (2020). *Language models are few-shot learners*. Advances in Neural Information Processing Systems, 33, 1877-1901.
-  Russell, S., & Norvig, P. (2020). *Artificial Intelligence: A Modern Approach* (4th ed.). Pearson.
-  LangChain. (2024). *LangGraph: Build stateful multi-agent applications*. <https://langchain-ai.github.io/langgraph/>
-  Google AI. (2024). *Gemini API Documentation*. <https://ai.google.dev/docs>
-  Streamlit Inc. (2024). *Streamlit Documentation*. <https://docs.streamlit.io/>

Thank You!